Java Full Stack Development

2. IntelliJ Debugging and Testing

Module Topics

- Debugging in IntelliJ
- Run Configurations
- Running and Managing Tests in IntelliJ

Debugger

- Logging tells you what happened
  - But the debugger shows you why
  - Logs record output at a point in time
  - The debugger exposes the full program state at any point in execution
  - Use cases in Spring Boot
    - Understanding exactly how Spring resolves a bean dependency
    - Inspecting the state of a request as it passes through a filter chain
    - Diagnosing a data mapping failure between a JSON payload and a Java object
    - Stepping through conditional logic to verify branching behaviour

```
1   package org.example;                                                    OFF
2
    To Run code, press Shift F10 or click the ▶ icon in the gutter.
5 ▶ public class Main {  new *
6 ▶     public static void main(String[] args) {  new *
            Press Alt Enter with your caret at the highlighted text to see how IntelliJ IDEA
            suggests fixing it.
9           System.out.printf("Hello and welcome!");
10
11          for (int i = 1; i <= 5; i++) {
                Press Shift F9 to start debugging your code. We have set one ●
                breakpoint for you, but you can always add more by pressing Ctrl 8.
●               System.out.println("i = " + i);
15          }
16      }
17  }
```

© 2026 by Innovation In Software Corporation

Debugger and Spring Boot

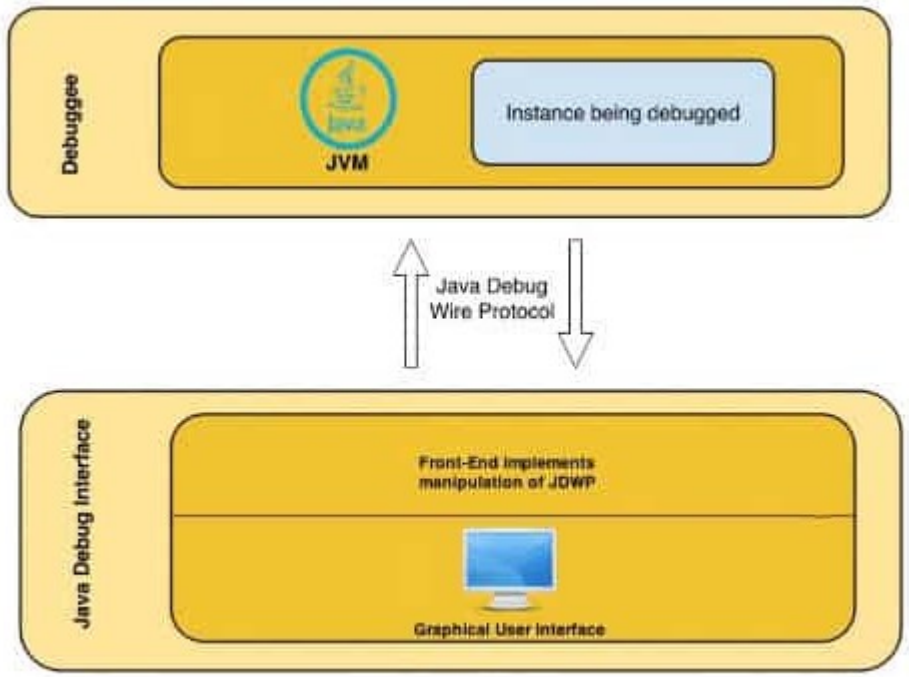
- Spring Boot framework
  - Does a significant amount of work before the code runs
    - Auto-configuration
    - Bean initialization
    - Dependency injection
    - Filter registration
  - Debugger lets you step into any of this framework code when needed
  - Particularly powerful when something fails silently during context startup

```
1   package org.example;                                                    OFF
2
    To Run code, press Shift F10 or click the ▶ icon in the gutter.
5 ▶ public class Main {  new *
6 ▶     public static void main(String[] args) {  new *
            Press Alt Enter with your caret at the highlighted text to see how IntelliJ IDEA
            suggests fixing it.
9           System.out.printf("Hello and welcome!");
10
11          for (int i = 1; i <= 5; i++) {
                Press Shift F9 to start debugging your code. We have set one ●
                breakpoint for you, but you can always add more by pressing Ctrl 8.
●               System.out.println("i = " + i);
15          }
16      }
17  }
```

© 2026 by Innovation In Software Corporation

IntelliJ Debugger

- IntelliJ debugger
  - Communicates with the JVM via the Java Debug Wire Protocol (JDWP)
    - JDWP is a standard JVM protocol, not IntelliJ-specific
    - The JVM exposes a socket that the debugger attaches to
  - Two attachment modes
    - Launch mode: IntelliJ starts the JVM and attaches automatically
    - Remote mode: IntelliJ attaches to an already-running JVM over a network socket



Java Platform Debug Architecture

Image Credit:https://strobecorp.com/java-debugging-tools/

© 2026 by Innovation In Software Corporation

IntelliJ Debugger

- IntelliJ debugger
  - The debugger operates on the compiled bytecode resolved against the source
    - Source mapping is automatic when source is available in the project
    - No special compiler flags are needed for standard Spring Boot projects

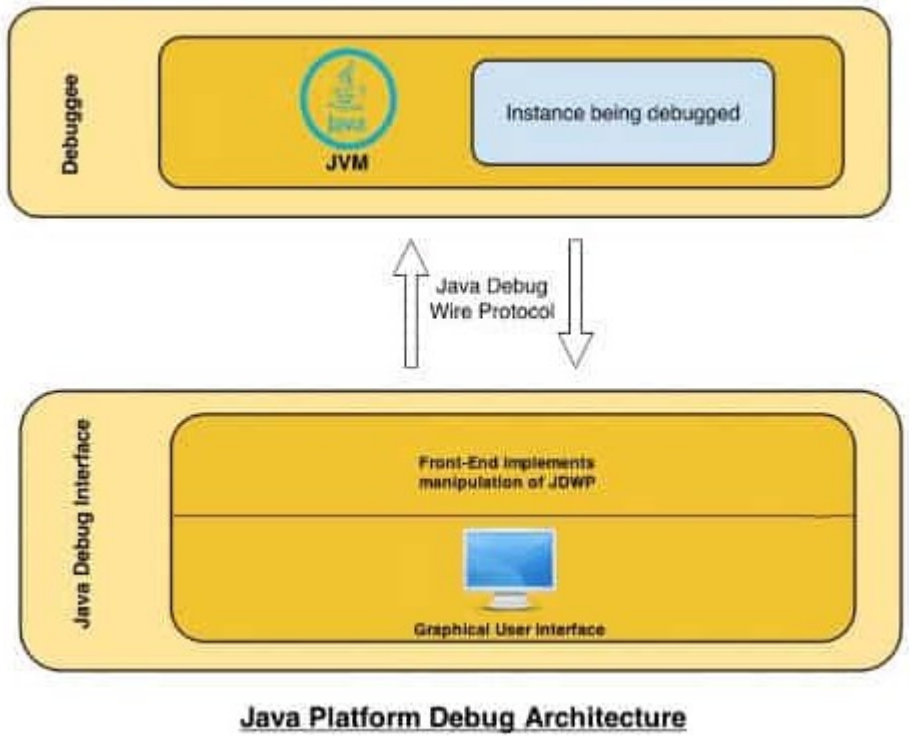
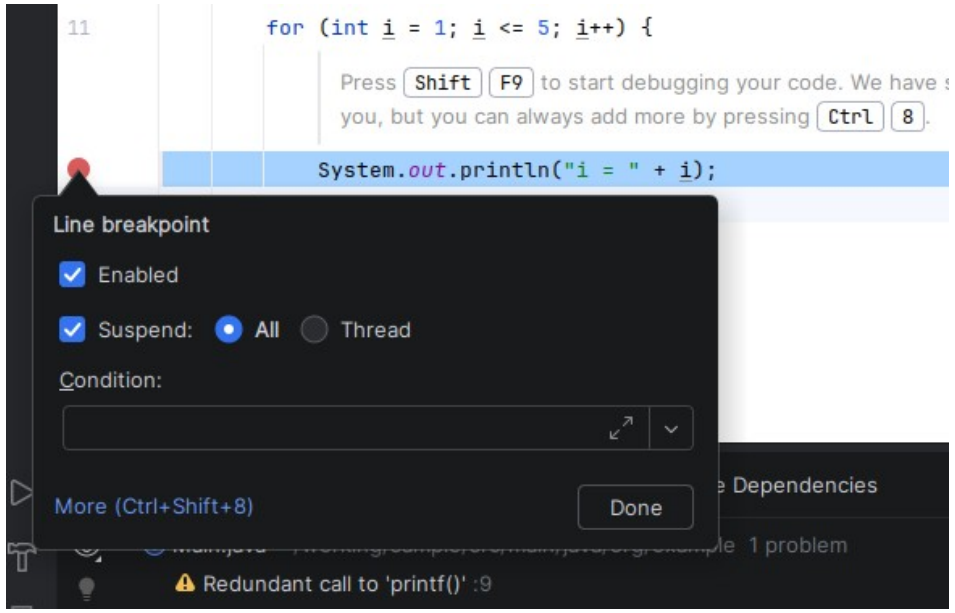


Image Credit:https://strobecorp.com/java-debugging-tools/

© 2026 by Innovation In Software Corporation

Breakpoints

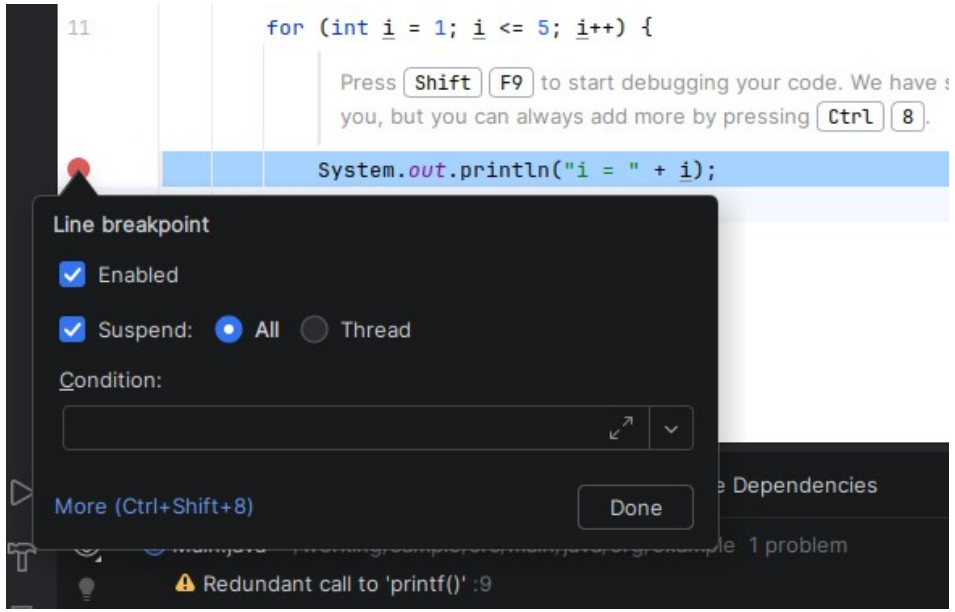
- An instruction to suspend the JVM at a specific location
- Setting a breakpoint
  - Click the gutter (the grey margin left of line numbers) on any executable line
  - Or Ctrl+F8 to toggle a breakpoint at the current cursor position



© 2026 by Innovation In Software Corporation

Breakpoints

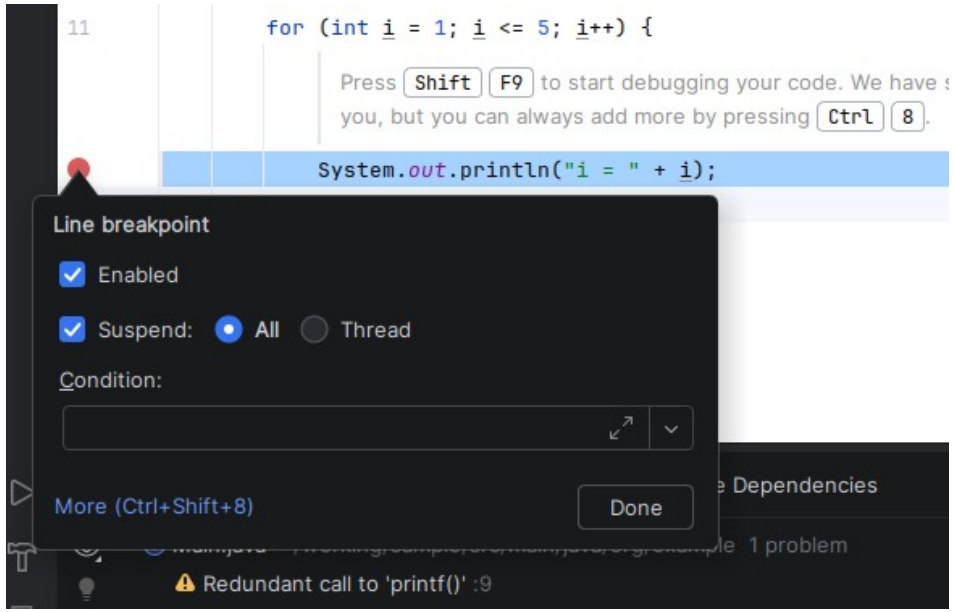
- Breakpoint types
  - Line breakpoint
    - Suspends execution when that line is reached
  - Method breakpoint
    - Suspends on entry, exit, or both for a method
  - Field watchpoint
    - Suspends when a specific field is read or written
  - Exception breakpoint
    - Suspends when a specific exception type is thrown



© 2026 by Innovation In Software Corporation

Breakpoints

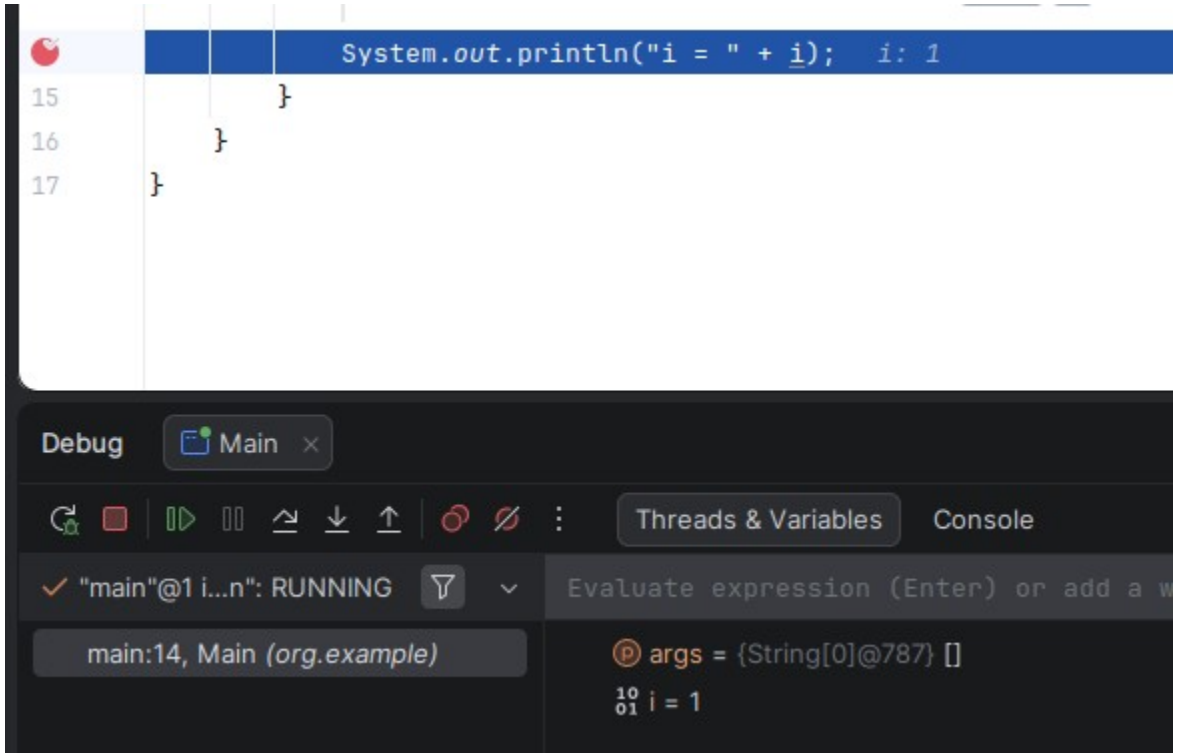
- Conditional breakpoints
  - Only pause when an expression evaluates to true
    - Right-click a breakpoint → set a condition
    - Example: orderId.equals("ORD-42") pauses only for that specific order
    - Avoids stopping on every iteration when debugging a loop
- Logging breakpoints
  - Print a message without halting execution
    - Right-click a breakpoint → uncheck "Suspend" → enable "Evaluate and log"
    - Effectively a zero-code, runtime System.out.println
    - Useful in production-like scenarios where stopping execution is not practical



© 2026 by Innovation In Software Corporation

Debug Tool Window

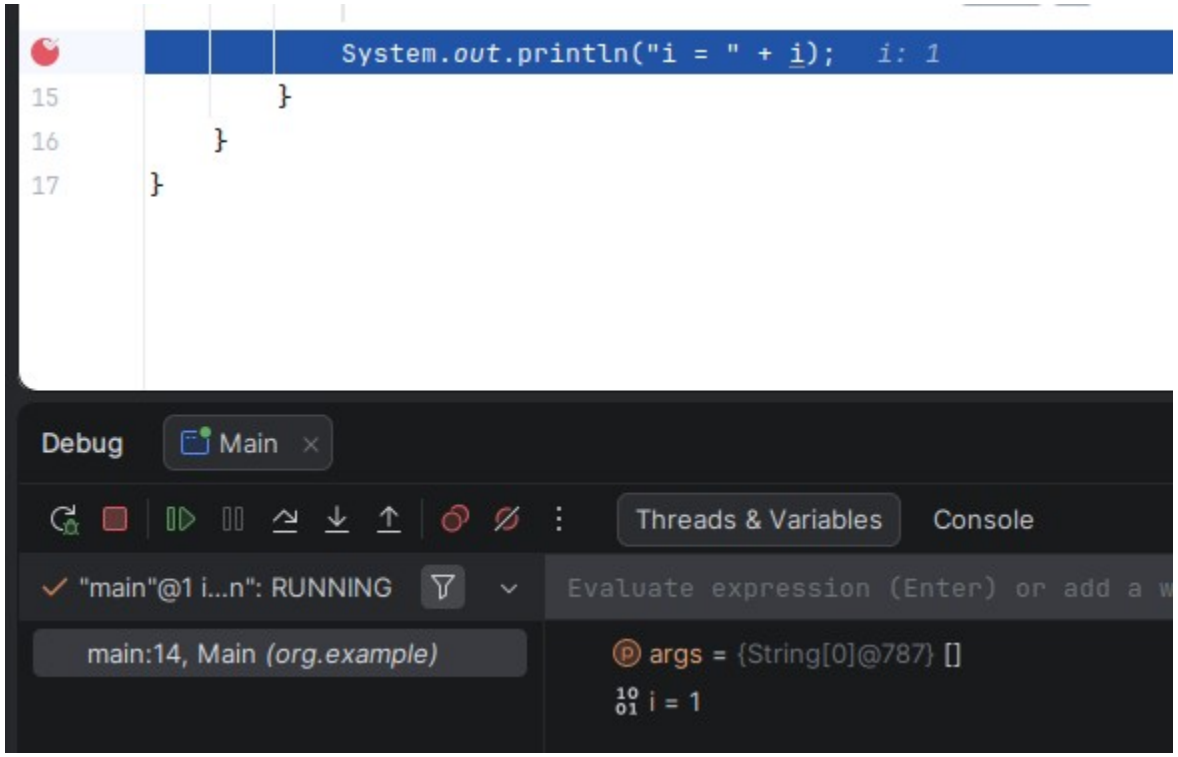
- The Debug tool window opens automatically when a debug session starts
- Key panels
  - Frames: the current call stack
    - Click any frame to inspect its local variables
  - Variables: all local variables and fields visible at the current stack frame
  - Watches: expressions evaluated continuously during the session
  - Console: stdout/stderr from the running JVM



© 2026 by Innovation In Software Corporation

Debug Tool Window

- Navigating the call stack
  - Every frame in the stack is clickable
  - Selecting a frame shows the variables in scope at that point in the call chain
  - Being able to click a frame and see the exact state of variables at that level
  - Helps answer the question "which Spring component did something unexpected before my code was called?"



© 2026 by Innovation In Software Corporation

Stepping Controls

- Uses
  - Step Over is the most common action during a debugging session
    - Move line by line through the current method without descending into calls
  - Step Into is used when you need to understand what a called method actually does.
  - Force Step Into
    - IntelliJ by default skips stepping into JDK and library code
    - If you need to trace into Spring's DispatcherServlet, a Jackson deserializer, or any framework class
    - Force Step Into bypasses that filter.

Action	Shortcut	Behaviour
Step Over	`F8`	Execute the current line; if it's a method call, run it without entering
Step Into	`F7`	Enter the method being called on the current line
Force Step Into	`Alt+Shift+F7`	Step into even when IntelliJ would normally skip it (e.g. library code)
Step Out	`Shift+F8`	Finish the current method and return to the caller
Run to Cursor	`Alt+F9`	Run until the cursor position, then suspend
Resume	`F9`	Continue execution until the next breakpoint

© 2026 by Innovation In Software Corporation

Evaluating Expressions

- Evaluate Expression: Alt+F8
  - Execute any valid Java expression in the context of the current stack frame
  - Calls methods, accesses fields, constructs objects
  - Results display immediately
- Inline variable values
  - IntelliJ renders current variable values directly on the source line, next to the code
  - Grey text showing the actual value, visible without opening the Variables panel
  - Quick Evaluate: hover over any expression while suspended
  - A tooltip shows the current evaluated value

© 2026 by Innovation In Software Corporation

Modifying State

- Variable values can be changed while the JVM is suspended
  - Right-click a variable in the Variables panel → Set Value
  - Allows testing "what if this value were different" without code changes
- Drop Frame (Rewind a method call)
  - Removes the current frame from the call stack
  - Re-executes the method from its beginning with the same entry state
  - Useful when you stepped past the point of interest
  - Not available across native frames or after side effects (DB writes, network calls)

© 2026 by Innovation In Software Corporation

Remote Debugging

- Remote debugging allows attaching IntelliJ to a JVM running outside the IDE
  - Docker containers, remote servers, or locally started processes
  - The target JVM must be started with JDWP enabled
    - -agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=*:5005
- In IntelliJ
  - Run → Edit Configurations → Add New → Remote JVM Debug
    - Configure host and port to match the target JVM
    - Connect using the Debug button (not Run)
- Common use cases
    - Debugging a Spring Boot app running inside a local Docker container
    - Diagnosing a startup failure in a pre-production environment

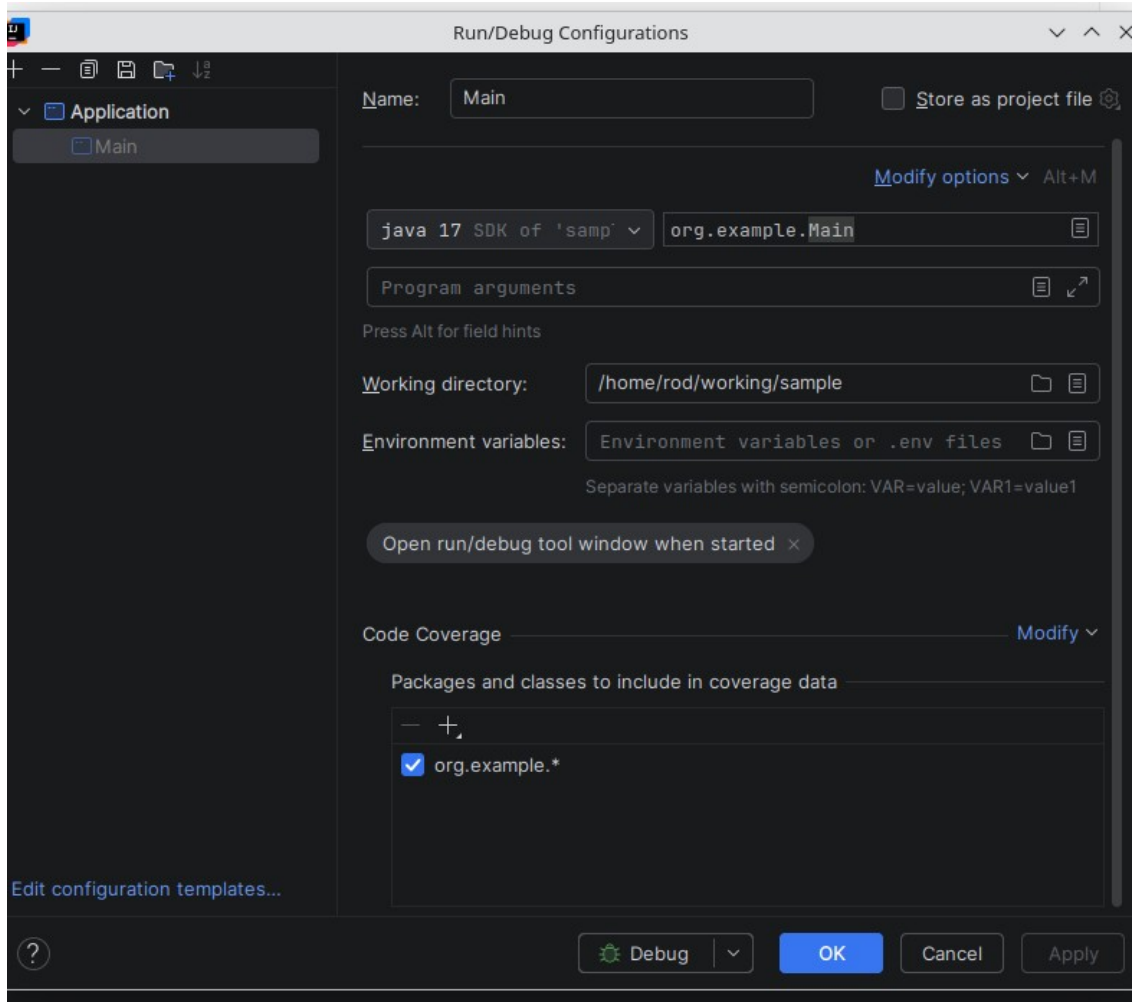
© 2026 by Innovation In Software Corporation

Run Configurations

© 2026 by Innovation In Software Corporation

Run Configuration

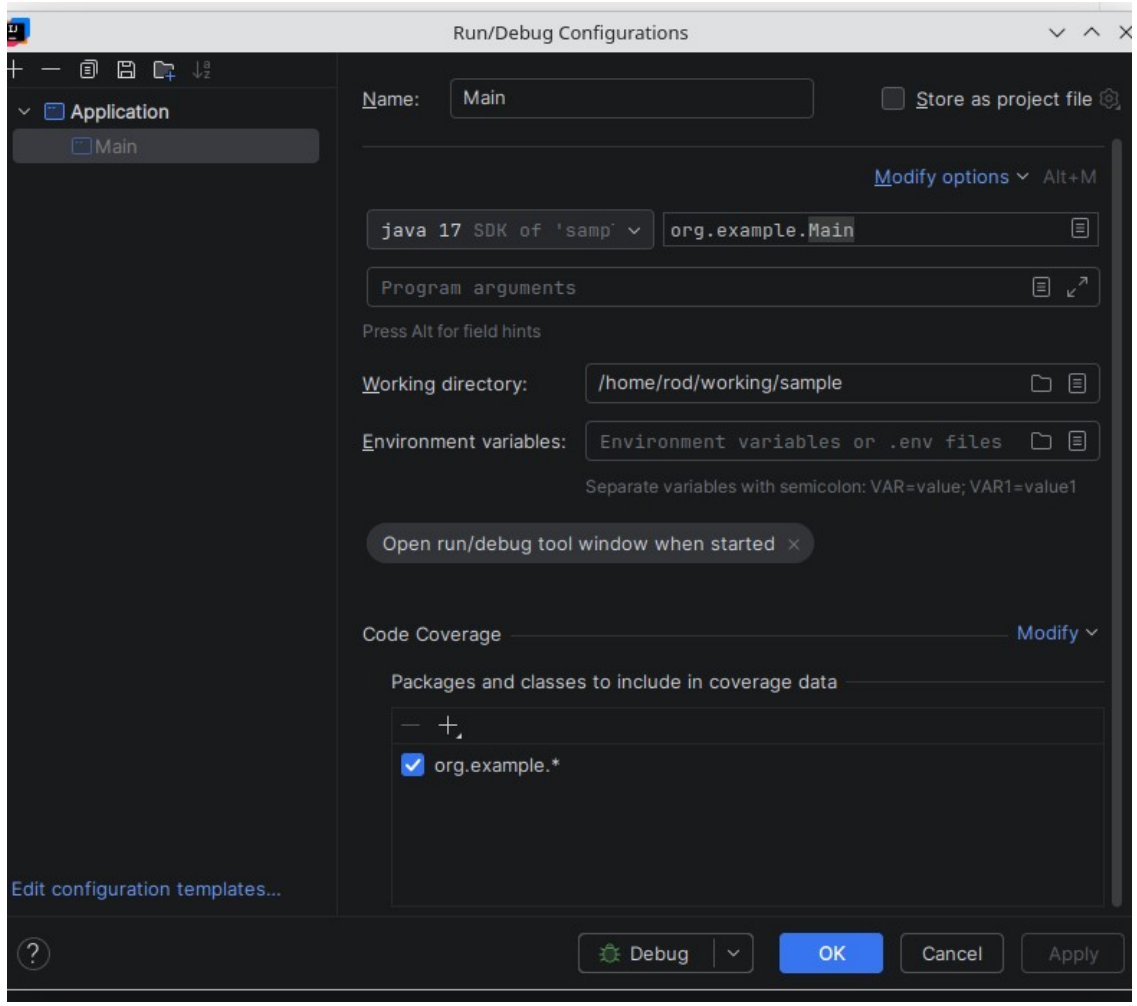
- Temporary configuration
  - Created when developer manually starts a Spring Boot application by clicking the green arrow on the main class
    - If they never save it, a teammate cloning the same repository will have no launch configuration at all
    - They must recreate it instead
  - Shared configurations stored in .idea/runConfigurations/ solve this
    - Every developer who clones the repository gets the same set of launch instructions
    - Including the correct active profile, environment variables, JVM arguments, and working directory
  - This is a team hygiene practice, not an individual productivity feature



© 2026 by Innovation In Software Corporation

Run Configuration

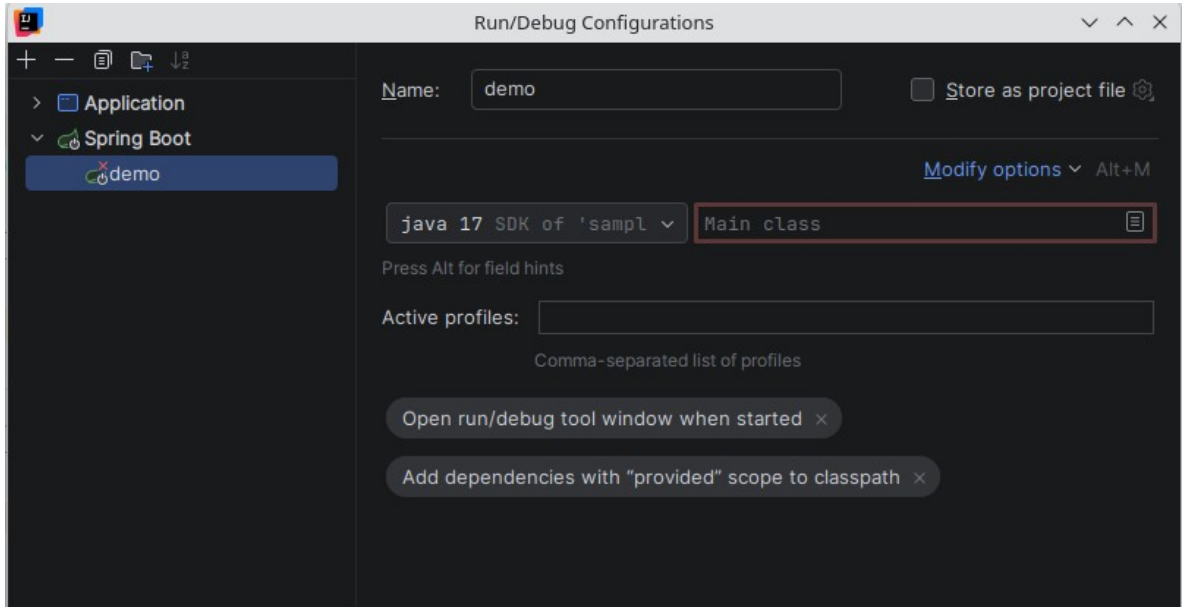
- Named, saved execution blueprint
  - Specifies
    - What to launch
    - How to launch it
    - With what environment
  - Every execution in IntelliJ uses a Run Configuration
- The three lifecycle states
  - Temporary: auto-created, not saved, eventually discarded
  - Permanent: saved to local IDE settings, survives restarts
  - Shared: committed to .idea/runConfigurations/ as XML, visible to the whole team
  - Shared configurations should be the standard on any team project



© 2026 by Innovation In Software Corporation

Spring Boot Run Configuration

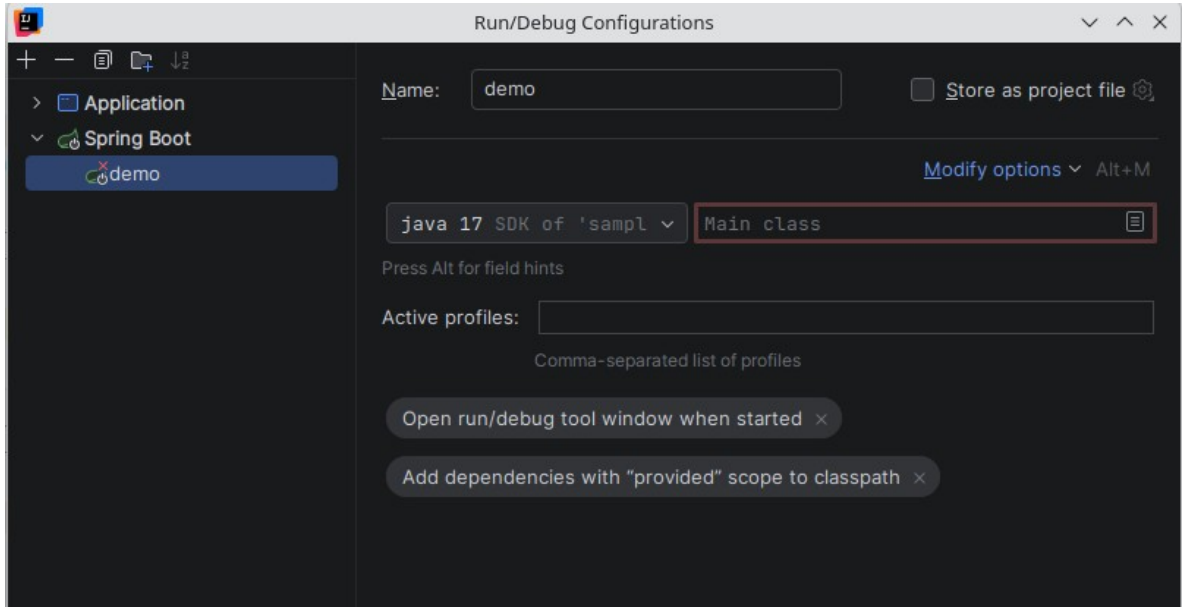
- Standard for launching Spring applications
  - Available in IntelliJ Ultimate (not Community)
  - Community edition uses the generic Application type instead
- Key fields:
  - Main class: the @SpringBootApplication entry point
  - Active profiles: sets spring.profiles.active (e.g., dev)
  - VM options: JVM flags, including JDWP for remote debug enablement
  - Environment variables: key-value pairs injected into the process environment
  - Working directory: the directory the JVM uses as its working root



© 2026 by Innovation In Software Corporation

Spring Boot Run Configuration

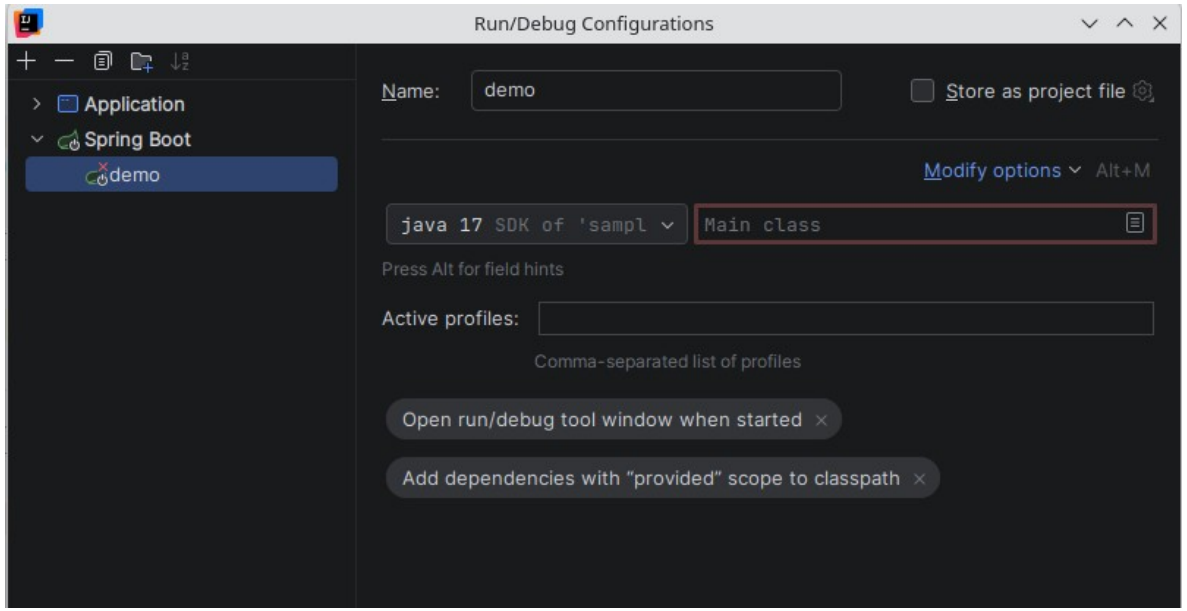
- A properly configured team project
  - Will have three shared configurations
    - One for local (using local infrastructure)
    - One for dev (pointing at shared dev services)
    - Possibly one with a test profile for integration testing
  - The Environment Variables field is the right place to supply secrets and connection strings that should not be committed to source control
    - These are injected into the process at startup and accessible via Spring's @Value annotation or Environment bean.



© 2026 by Innovation In Software Corporation

Spring Boot Run Configuration

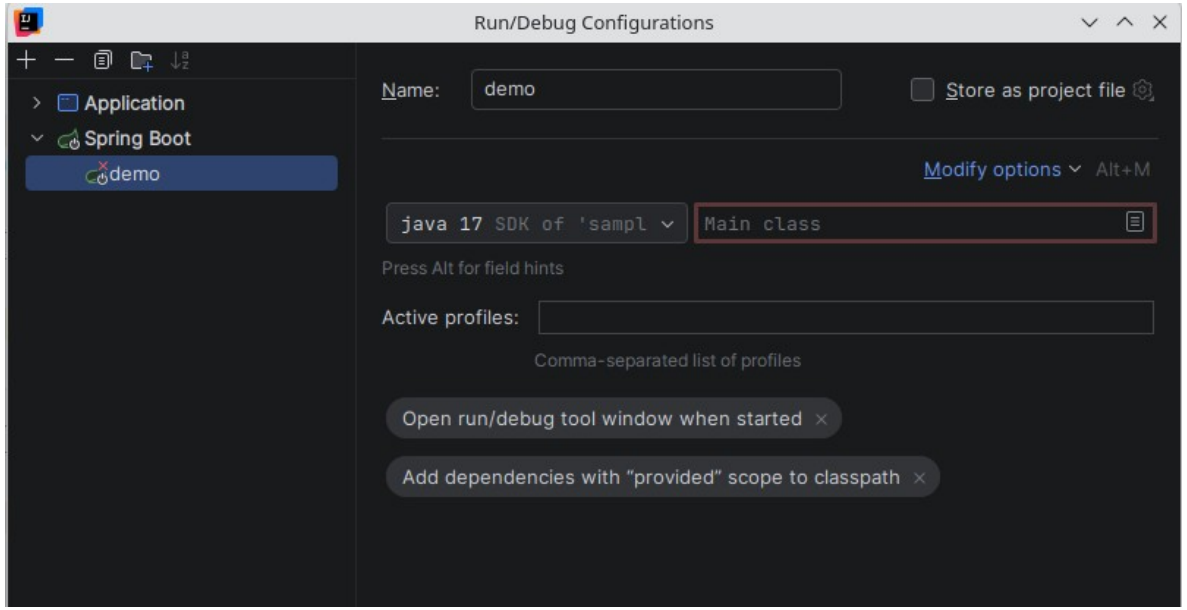
- VM options examples
  - -Xmx2g maximum heap for the application JVM (separate from IDE heap)
  - -Dspring.datasource.url=jdbc:oracle:thin:@localhost:1521:XE override a Spring property
  - -agentlib:jdwp=... enable remote debugging for a separately launched process
- Environment Variables
  - Injected into the OS environment of the process
  - Accessed in Spring via ${ENV_VAR_NAME} in YAML or @Value("${ENV_VAR_NAME}")
  - Preferred for secrets because they are not visible in log output by default



© 2026 by Innovation In Software Corporation

Spring Boot Run Configuration

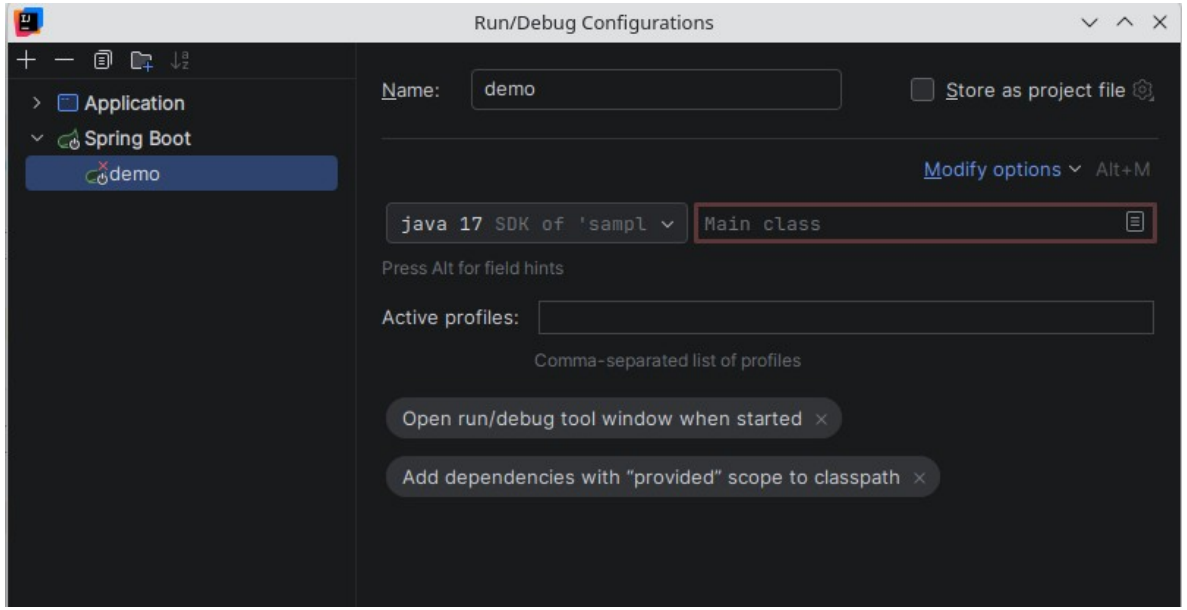
- A common mistake
  - Attempting to pass environment variables as VM options or vice versa
  - Spring's Environment abstraction resolves property values in a defined order of precedence
  - Environment variables sit higher in the precedence chain than application.yml values but lower than command-line arguments.
  - VM options set as system properties override application.yml values



© 2026 by Innovation In Software Corporation

Spring Boot Run Configuration

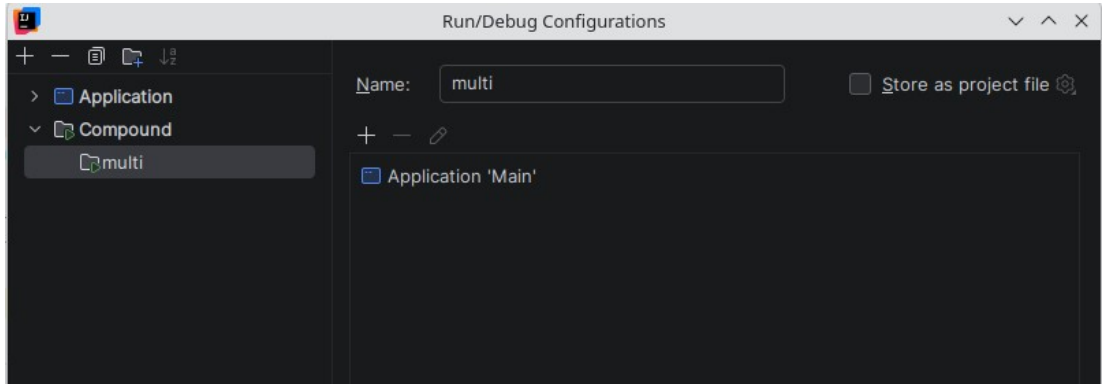
- A common mistake
  - The full order, from highest to lowest priority in a typical Spring Boot application
    - command-line args
    - OS environment variables
    - Java system properties
    - application-{profile}.yml
    - application.yml.



© 2026 by Innovation In Software Corporation

Compound Run Configuration

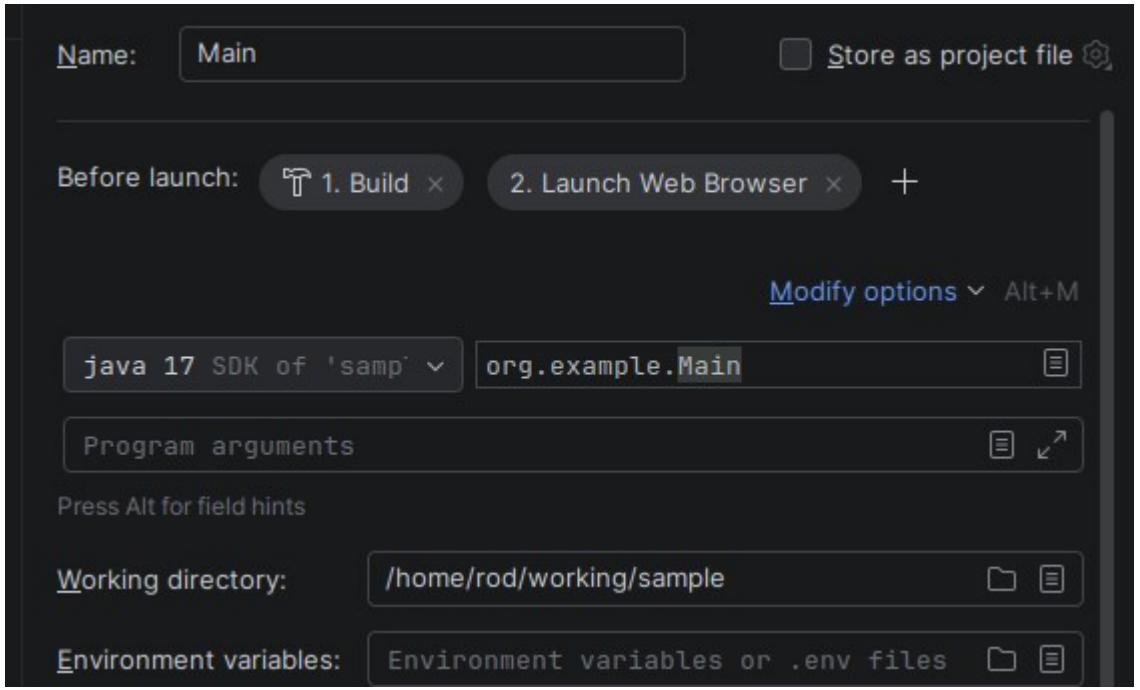
- A Compound configuration
  - Lunches multiple Run Configurations simultaneously
  - Single click starts all configured services together
  - Common use cases
    - Start the Spring Boot backend and a local mock OAuth server together
    - Launch multiple microservices that the current service depends on
    - Start the backend and a local Kafka instance before running integration tests
  - Creating a Compound configuration:
    - Run → Edit Configurations → + → Compound
    - Add individual configurations to the Compound



© 2026 by Innovation In Software Corporation

Before Launch Actions

- Every Run Configuration supports a Before Launch list
  - Actions executed automatically before the main process starts
  - Common Before Launch actions
    - Build: compiles the project (enabled by default)
    - Run Maven Goal: execute clean package, test, or any Maven lifecycle phase
    - Run Another Configuration: start a dependency service first
    - Run External Tool: execute a shell script, Docker command, or any CLI tool
  - Before Launch actions run sequentially, in order



© 2026 by Innovation In Software Corporation

Running and Managing Tests in IntelliJ

© 2026 by Innovation In Software Corporation

Discovering Tests

- IntelliJ scans the src/test/java source root for test classes
  - Test classes annotated with @Test (JUnit 5) or extending TestCase (JUnit 4) are recognized
  - Spring Boot test classes annotated with @SpringBootTest are also recognized
- Test discovery is automatic
  - No registration required
  - The test source root is configured in the Project Structure (Ctrl+Alt+Shift+S)
  - Maven projects set this automatically from the pom.xml configuration

AppTest.java

```
1   package com.mycompany.app;
2
3   import ...
6
7   /**
8    * Unit test for simple App.
9    */
10  public class AppTest    Rod Davison +1
11  {
12      /**
13       * Rigorous Test :-)
14       */
15      @Test    Rod Davison +1
16      public void shouldAnswerWithTrue() { assertTrue( condition: true ); }
20  }
21
```

© 2026 by Innovation In Software Corporation

Discovering Tests

- When tests are not being discovered
  - Often because the test source folder (src/test/java) is not marked as a Test Sources Root in IntelliJ's project model
  - In an imported Maven project this is automatic
    - Maven's standard directory layout is understood by IntelliJ's Maven import
  - If tests are missing from the test runner, check
    - Project Structure → Modules → Source Folders
    - Verify the test folder has the green test icon, not the blue source icon
    - A manual Maven reload usually fixes this by re-importing the project model from the pom.xml

AppTest.java

```
1   package com.mycompany.app;
2
3   import ...
6
7   /**
8    * Unit test for simple App.
9    */
10  public class AppTest    Rod Davison +1
11  {
12      /**
13       * Rigorous Test :-)
14       */
15      @Test    Rod Davison +1
16      public void shouldAnswerWithTrue() { assertTrue( condition: true ); }
20  }
21
```

© 2026 by Innovation In Software Corporation

Running Tests

- Run options, from most specific to most general
  - Single test method
    - Click the green play icon in the gutter next to an @Test method
  - Single test class
    - Click the green play icon in the gutter next to the class declaration
  - All tests in a package
    - Right-click the package in the Project window → Run Tests
  - All tests in the module
    - Right-click the src/test/java root → Run All Tests
  - Keyboard shortcut: Ctrl+Shift+F10
    - Run the test at the current cursor position

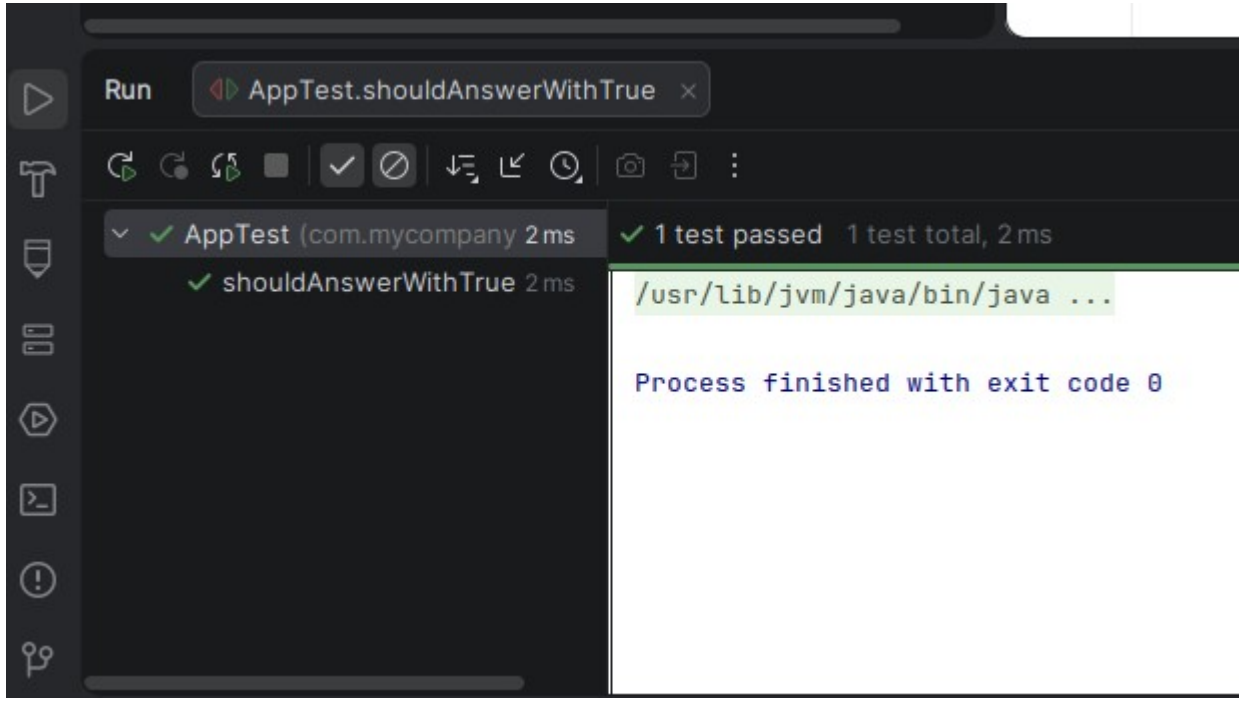
AppTest.java

```
1   package com.mycompany.app;
2
3   import ...
6
7   /**
8    * Unit test for simple App.
9    */
10  public class AppTest    Rod Davison +1
11  {
12      /**
13       * Rigorous Test :-)
14       */
15      @Test    Rod Davison +1
16      public void shouldAnswerWithTrue() { assertTrue( condition: true ); }
20  }
21
```

© 2026 by Innovation In Software Corporation

The Test Run Tool Window

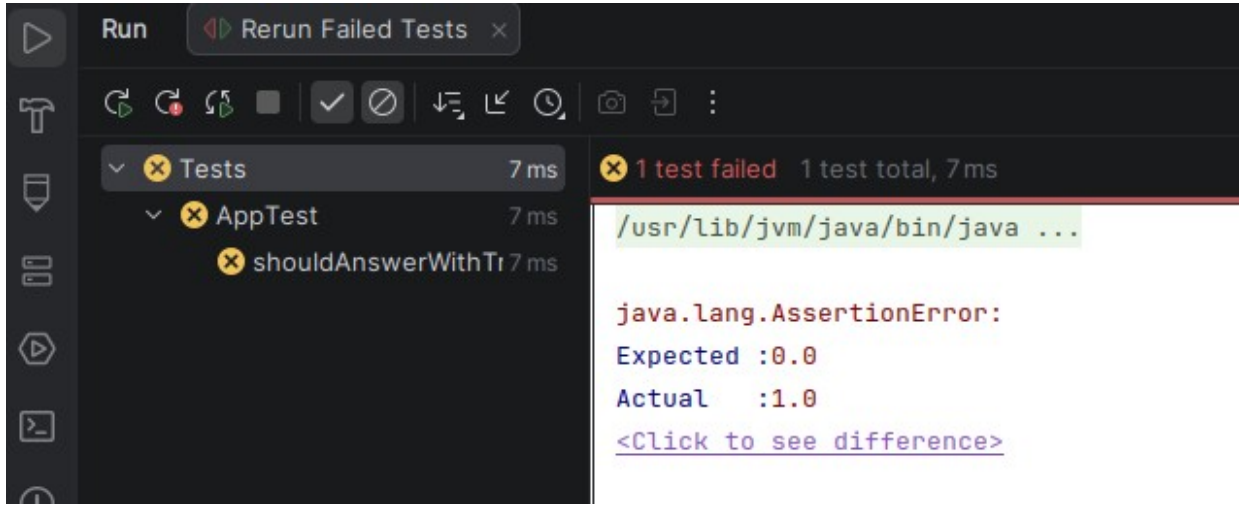
- Opens automatically when tests execute
  - Result structure:
    - Tests are displayed in a tree: class → method
    - Green checkmark: test passed
    - Red X: test failed (assertion failure or unexpected exception)
    - Yellow: test ignored or skipped (@Disabled)



© 2026 by Innovation In Software Corporation

The Test Run Tool Window

- Failure details
  - Click any failed test to see the assertion error message
  - The diff view shows expected vs actual values side by side for assertion failures
  - The full stack trace is available with clickable frames that navigate to source



© 2026 by Innovation In Software Corporation

Debugging Tests

- Can be launched in debug mode exactly like application code
  - Click the debug icon (bug) instead of the run icon next to a test method or test class
  - Or right-click → Debug
- Breakpoints set in test code and in production code are both honoured
  - Set a breakpoint in the test method to inspect test setup
  - Set a breakpoint in the production method under test to inspect its execution
  - Conditional breakpoints work the same as in application debugging
  - The Debug tool window shows the test execution call stack

© 2026 by Innovation In Software Corporation

Debugging Tests

- Debugging tests
  - Effective way to understand unexpected test behaviour
  - If a test is failing with a confusing assertion message
    - Setting a breakpoint immediately before the assertion
    - Evaluating the actual value using Alt+F8 is usually faster than adding logging and rerunning

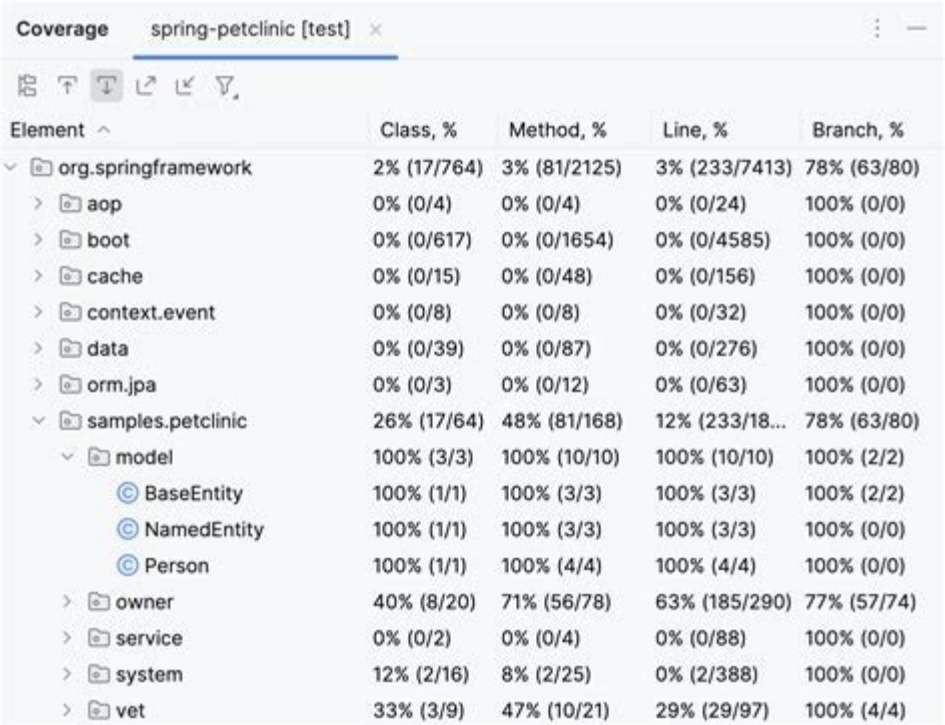
```
9    * unit test for simple App.
10   */
11   public class AppTest    Rod Davison *
12   {
13       /**
14        * Rigorous Test :-)
15        */
16       @Test    Rod Davison *
17
18
19                                                  0.0,  delta: .00);
20
21
22
```

Run 'shouldAnswerWithTrue()' Ctrl+Shift+F10
Debug 'shouldAnswerWithTrue()'
Run 'shouldAnswerWithTrue()' with Coverage
Modify Run Configuration...

© 2026 by Innovation In Software Corporation

Test Coverage

- IntelliJ can measure code coverage while running tests
  - Right-click a test class or package → Run with Coverage
  - Or use the Run with Coverage button in the toolbar
- Coverage results appear in
  - The editor gutter
    - Green (covered) / red (not covered) line highlighting
  - The coverage tool window
    - Class-level and method-level coverage percentages
  - A full HTML report (exportable)
- Coverage modes
  - IntelliJ coverage: faster, built-in
  - JaCoCo: the standard in CI pipelines; matches what Maven/Surefire reports



Coverage | spring-petclinic [test]

Element	Class, %	Method, %	Line, %	Branch, %
org.springframework	2% (17/764)	3% (81/2125)	3% (233/7413)	78% (63/80)
aop	0% (0/4)	0% (0/4)	0% (0/24)	100% (0/0)
boot	0% (0/617)	0% (0/1654)	0% (0/4585)	100% (0/0)
cache	0% (0/15)	0% (0/48)	0% (0/156)	100% (0/0)
context.event	0% (0/8)	0% (0/8)	0% (0/32)	100% (0/0)
data	0% (0/39)	0% (0/87)	0% (0/276)	100% (0/0)
orm.jpa	0% (0/3)	0% (0/12)	0% (0/63)	100% (0/0)
samples.petclinic	26% (17/64)	48% (81/168)	12% (233/18...	78% (63/80)
model	100% (3/3)	100% (10/10)	100% (10/10)	100% (2/2)
BaseEntity	100% (1/1)	100% (3/3)	100% (3/3)	100% (2/2)
NamedEntity	100% (1/1)	100% (3/3)	100% (3/3)	100% (0/0)
Person	100% (1/1)	100% (4/4)	100% (4/4)	100% (0/0)
owner	40% (8/20)	71% (56/78)	63% (185/290)	77% (57/74)
service	0% (0/2)	0% (0/4)	0% (0/88)	100% (0/0)
system	12% (2/16)	8% (2/25)	0% (2/388)	100% (0/0)
vet	33% (3/9)	47% (10/21)	29% (29/97)	100% (4/4)

Image Credit: https://www.jetbrains.com.cn/en-us/help/idea/code-coverage.html

© 2026 by Innovation In Software Corporation

Questions?